

Formalização da correção do algoritmo Bubblesort

Aquila Macedo Costa – 202021800
Douglas Nunes da Costa – 200016997
Rafael Oliveira Bonach – 221008365

7 de julho de 2026

Este trabalho apresenta uma prova formal da correção do algoritmo de ordenação por borbulhamento (a função *bs* a seguir). A formalização foi feita no assistente de provas Coq. O assistente de provas Coq utiliza o sistema de Dedução Natural, o que o torna adequado para o desenvolvimento de atividades computacionais no curso de Lógica Computacional 1. O Coq permite a extração de código certificado em diversas linguagens funcionais, como Ocaml, Haskell e Scheme. Nesse projeto, também será disponibilizado o arquivo haskell extraído no Coq.

Iniciaremos definindo a função *bubble* que recebe uma lista de naturais como argumento, e percorre esta lista comparando elementos consecutivos. Chamamos este processo de borbulhamento:

```
Function bubble (l: list nat) {measure length l} :=  
  match l with  
  | nil => nil  
  | x::nil => x::nil  
  | x::y::l =>  
    if x <= y  
    then x::(bubble (y::l))  
    else y::(bubble (x::l))  
  end.
```

Observe que esta função não é estruturalmente recursiva porque, por exemplo, a lista $(x::l)$ não é uma sublista da lista original $(x::y::l)$. Neste caso, utilizamos **Function** para construir esta função e precisamos fornecer a medida que decresce em cada chamada recursiva, além de provar que esta medida efetivamente decresce a cada chamada recursiva. Por exemplo, *bubble* $(2::1::nil)$ retorna a lista $(1::2::nil)$.

Eval compute in *bubble* $(2::1::nil)$.

```
= 1 :: 2 :: nil  
: list nat
```

Entretanto, conseguimos observar a não recursividade da função *bubble* a seguir. Em um exemplo onde a lista é $(3::2::1::nil)$, a função retorna como resultado $(2::1::3::nil)$, que apesar de ser uma permutação da lista original, ela não é o resultado ordenado da lista.

Eval compute in *bubble* $(3::2::1::nil)$.

```
= 2 :: 1 :: 3 :: nil  
: list nat
```

A função principal, ou seja, o algoritmo bubble sort propriamente dito, é dada pela função *bs* abaixo que recebe uma lista de naturais como argumento:

```
Fixpoint bs (l: list nat) :=
```

```

match l with
| nil ⇒ nil
| h::l' ⇒ bubble (h::(bs l'))
end.

```

Neste caso sim, a função *bs* deve ser capaz de ordenar a lista original. Entretanto, precisamos realizar algumas provas para garantir que de fato, *bs* seja capaz de causar a ordenação de uma lista.

1 Provas da Ordenabilidade de *bs*

1.1 Provando permutação

Primeiro, como havíamos demonstrado, precisamos provar que o algoritmo é capaz de gerar uma permutação da lista original. Para isso, provaremos os lemas a seguir.

1.1.1 Lema 1

O primeiro lema que precisamos provar é que a função *bubble* gera uma permutação da lista original:

Lemma *bubble_perm*: $\forall l, \text{Permutation } l \text{ (bubble } l\text{)}.$

Para provar esse lema, será necessário provar 4 casos fundamentais:

- 1. Permutação $() = \text{bubble } ()$:
Esse caso é provado trivialmente, dado que a permutação de uma lista vazia é própria lista vazia. O algoritmo *bubble* retorna uma lista vazia.
- 2. Permutação $(x :: \text{nil}) = \text{bubble } (x :: \text{nil})$:
Esse caso também é provado trivialmente, dado que a permutação de uma lista com um único elemento é própria lista. O algoritmo *bubble* retorna a lista com o único elemento.
- 3. Permutação $(x :: y :: l0) = \text{bubble } (x :: y :: l0)$:

Esse caso, como a nossa hipótese de indução é dada por *Permutation* $(y :: l0) \text{ (bubble } (y :: l0))$, precisamos utilizar a regra de permutação *perm_skip*, que diz que a permutação de uma lista com um elemento constante é equivalente a permutação da sublista. Assim, podemos aplicar a hipótese de indução *IHl0* para provar que a sublista *l0* é uma permutação da lista *bubble l0*.

- 4. Permutação $(y :: x :: l0) = \text{bubble } (y :: x :: l0)$:

Para provarmos esse caso, precisaremos utilizar a regra de permutação *perm_trans* com a lista $(y :: x :: l0)$. Essa regra diz que se uma lista é uma permutação de outra, e essa segunda lista é uma permutação de uma terceira, então a primeira lista é uma permutação da terceira. Isso nos dá duas ramificações a serem provadas:

- *Permutation* $(x :: y :: l0) \text{ (} y :: x :: l0 \text{)}$:
Para a primeira ramificação dessa prova, lista $(y :: x :: l0)$ é uma permutação da lista $(x :: y :: l0)$, podemos aplicar a regra de permutação *perm_swap* que diz que a permutação de uma lista com dois elementos é equivalente a permutação da lista com os elementos trocados de posição, concluindo essa ramificação.
- *Permutation* $(y :: x :: l0) \text{ (} y :: \text{bubble } (x :: l0) \text{)}$:
Depois, provamos a segunda ramificação, lista $(x :: y :: l0)$ é uma permutação da lista *bubble* $(x :: l0)$, utilizando a regra de permutação *perm_skip* e aplicando a hipótese de indução *IHl0* para provar que a sublista *l0* é uma permutação da lista *bubble l0*.

Assim, concluímos a prova do lema 1, que diz que a função *bubble* retorna uma permutação da lista original.

1.1.2 Lema 2:

Agora, vamos provar que a função recursiva *bs* retorna uma permutação da lista original através do seguinte lema:

Lemma *bs_perm*: $\forall l, \text{Permutation } l \ (bs\ l).$

Para provar esse lema, será necessário provar 2 casos:

- 1. Permutação $() = bs\ ()$:

Esse caso é provado trivialmente, dado que a permutação de uma lista vazia é própria lista vazia. O algoritmo *bs* retorna uma lista vazia.

- 2. Permutação $(h :: tl) = bs\ (h :: tl)$:

Para provar esse caso, precisaremos utilizar a regra de permutação *perm_trans* com a lista $(h :: (bs\ tl))$. Essa regra diz que se uma lista é uma permutação de outra, e essa segunda lista é uma permutação de uma terceira, então a primeira lista é uma permutação da terceira. Com isso, teremos duas ramificações a serem provadas:

- 1º $Perm\ (h :: tl) = (h :: (bs\ tl))$:

Para provar essa ramificação, podemos aplicar a regra de permutação *perm_skip* que diz que a permutação de uma lista com um elemento constante é equivalente a permutação da sublista. Em seguida, basta usar a hipótese de indução *IHtl*, permutação da sublista *tl* é equivalente a permutação da lista *bs tl*.

- 2º $Perm\ (h :: (bs\ tl)) = bs\ (h :: tl)$:

Para provar essa ramificação, basta utilizar o lema 1 *bubble_perm*, que diz que a função *bubble* retorna uma permutação da lista original.

Assim, concluímos a prova do lema 2, que diz que o algoritmo recursivo *bs* retorna uma permutação da lista original.

1.2 Provando Ordenação

Agora que provamos que o algoritmo bubblesort consegue retornar uma permutação da lista original, precisamos provar que o algoritmo é capaz de ordenar a lista.

1.2.1 Lema 3:

Primeiro, precisamos provar que, para uma lista ordenada, a função *bubble* é capaz de mantê-la ordenada, ou seja, garantir a estabilidade de *bubble*. Para isso, utilizaremos o lema a seguir:

Lemma *bubble_stability*: $\forall l, \text{Sorted } le\ l \rightarrow bubble\ l = l.$

Essa prova é realizada por indução sobre a função *bubble*, fazendo com que a demonstração seja dividida em quatro casos.

- 1º - Lista vazio (*nil*):

Essa prova é trivial pois a função *bubble* retorna a lista vazia, que é igual a lista vazia.

- 2º - Possui 1 elemento ($x :: nil$):

Também é trivial, pois se a função *bubble* receber uma lista com um único elemento, ela retorna essa mesma lista, que é equivalente a lista ordenada.

- 3º - Possui pelo menos 2 elementos ($x :: y :: l0, x \leq y$):

Nesse caso, a lista não realizará a troca entre os dois elementos e prossegue recursivamente para a sublista $y :: l0$, assim temos que $x :: \text{bubble}(y :: l0) = x :: y :: l0$. Como a lista permanece ordenada, tomamos por hipótese de indução $\text{bubble}(y :: l0) = y :: l0$. Assim, aplicando a h.i., temos que $x :: y :: l0 = x :: y :: l0$, que é verdade.

Com isso, provamos que para uma lista ordenada, a função *bubble* retorna a mesma lista ordenada.

- 4º - Possui pelo menos 2 elementos ($x :: y :: l0, x > y$):

Esse é um caso particular, pois a forma que o nosso lema foi estruturado torna o caso impossível de ser provado assumindo que a lista recebida por *bubble* esteja ordenada. Inicialmente, temos a premissa $(x \leq y) = \text{false}$, ou seja, $x > y$. Assim, a função *bubble* retornará $y :: \text{bubble}(x :: l0)$. Entretanto, o passo indutivo consiste em provar $\text{Sorted le}(x :: y :: l0) \rightarrow y :: \text{bubble}(x :: l0) = x :: y :: l0$. Desta forma, utilizando a nossa premissa booleana, chegamos a uma contradição, invalidando esse caso com prova para a ordenação de uma lista ordenada utilizando o algoritmo *bubble*.

1.2.2 Lema 4:

Agora, precisamos provar uma característica fundamental da função *bubble*, que ela consegue fazer com que o maior elemento de uma lista torne-se o último elemento. Para isso, usaremos o lema a seguir:

Lemma *bubble_one*: $\forall x \ l, \text{Sorted le } l \rightarrow \text{Sorted le}(\text{bubble}(x :: l))$.

O lema *bubble_one* é provado através de duas ramificações:

- 1º - $() \rightarrow \text{bubble}(x :: ())$:

Iniciamos introduzindo o primeiro termo da implicação, e então utilizamos a regra *Sorted_cons* para abrir a nossa lista ordenada em uma lista vazia, que é provada trivialmente com *Sorted_nil*, e a relação de um termo com o vazio, que resultantemente dá o termo. Assim provamos que para uma lista com um único termo, o teorema é válido.

- 2º - $(h :: tl) \rightarrow \text{bubble}(x :: h :: tl)$:

Iniciamos da mesma forma que o ramo anterior. Entretanto, para que esse lado funcione, é necessário abrir a função *bubble* para dividir em duas novas ramificações. Essa abertura é necessária para tirarmos do *bubble* o primeiro elemento da lista. Isso divide a nossa árvore de provas em duas ramificações:

- $\text{Sorted le}(x :: \text{bubble}(h :: tl))$, para $(x \leq h)$:

Nessa ramificação, conseguimos utilizar a hipótese que $h < tl$ que é resultante da hipótese que tl está ordenado para abrirmos mais uma vez a nossa premissa em $\text{Sorted le}(\text{bubble}(h :: tl))$ e $x < (\text{bubble}(h :: tl))$. Assim, utilizamos a nossa o lema *bubble_stability* para provar que $\text{bubble}(h :: tl) = h :: tl$, e então conseguimos provar que $\text{Sorted le}(h :: tl)$.

Em seguida, utilizamos novamente a regra *bubble_stability* para termos as duas ramificações, $x < (h :: tl)$, que é provado trivialmente pelas hipóteses; $\text{Sorted le}(h :: tl)$, que também é provado trivialmente.

- $\text{Sorted le}(h :: \text{bubble}(x :: tl))$, para $(x > h)$:

Para essa ramificação, precisaremos estruturar uma prova para a calda da lista *tl*. Para isso, generalizamos a nossa prova para todos os tipos de calda de lista e provamos para cada tipo de calda:

- * Caso a calda seja vazia ($tl = \text{nil}$):

Nesse caso, precisamos provar que $(h :: \text{bubble}(x :: \text{nil}))$ está ordenado. Para isso, utilizamos a regra *Sorted_cons* para abrirmos a nossa lista ordenada e provamos que $\text{bubble}(x :: \text{nil}) = x :: \text{nil}$. Em seguida, usamos a função *bubble* que nos retorna $h \leq x$, gerando assim uma contradição nessa ramificação, uma vez que temos como premissa $h < x$.

- * Caso a calda não seja vazia ($tl = l$):
Nesse caso, teremos de entrar mais uma vez na função *bubble*, criando outras duas ramificações:
 - $(h :: x :: \text{bubble } (n :: l))$, para $(x \leq n)$:
Nessa ramificação, utilizamos a regra *Sorted_cons* para tirarmos h da lista, já que ele já está ordenado e então usamos a hipótese de indução para provar que $(x :: \text{bubble } (n :: l))$ está ordenado. Em seguida, provamos que o elemento $h \leq (x :: \text{bubble } (n :: l))$, que gera uma contradição com a premissa que $h < x$.
 - $(h :: n :: \text{bubble } (x :: l))$, para $(x > n)$:
Nessa ramificação, repetimos os mesmo passos da ramificação anterior, com a diferença que agora de fato $h \leq (n :: \text{bubble } (x :: l))$, condizendo com a nossa premissa $h \leq n$.

Desta forma, provamos que a função *bubble* consegue fazer com que o maior elemento de uma lista torne-se o último elemento.

1.2.3 Lema 5:

Por fim, provamos que o algoritmo *bs* ordena uma lista com o lema a seguir:

Lemma *bs_sorted*: $\forall l, \text{Sorted } le \ (bs \ l)$.

Essa prova é uma prova simples por utilizar o lema anterior *bubble_one*. Para realizar essa prova, precisamos fazer indução em l . Isso nos dá 2 casos principais para provar:

- 1° - *bs* ordena uma lista vazia:
É provado trivialmente utilizando a regra de ordenação *Sorted_nil*, ou seja, uma lista vazia está ordenada.
- 2° - Ordenar 1 termo em uma lista ordenada ($\text{bubble } (h :: bs \ tl)$):
Nesse caso, usamos o lema *bubble_one* para provar que o elemento maior se move para a posição correta. Isso resulta em *Sorted (bs l)*, ou seja, o resultado de *bs* está ordenado, que é provado pela nossa Hipótese de indução *IHtl*, que diz que *bs* ordena a sublista tl .

Com isso, provamos que a nossa função *bs* ordena uma lista de naturais, ou seja, a função *bs* retorna uma lista ordenada.

1.3 Teorema Final

Por fim, a correção do algoritmo *bs* é obtida pelo teorema a seguir que estabelece que o algoritmo *bs* retorna uma permutação da lista de entrada que está ordenada:

Theorem *bs_correto*: $\forall l, \text{Sorted } le \ (bs \ l) \wedge \text{Permutation } l \ (bs \ l)$.

A prova desse teorema é bem simples. Primeiro quebramos o nosso sequente em duas ramificações:

- 1° - Provar que *bs* ordena a lista de entrada:
Para isso, utilizamos o lema *bs_sorted*, que diz que a função *bs* ordena uma lista de naturais.
- 2° - Provar que *bs* retorna uma permutação da lista de entrada:
Para isso, utilizamos o lema *bs_perm*, que diz que a função *bs* retorna uma permutação da lista original.

Assim, provamos que o algoritmo *bs* retorna uma permutação da lista de entrada que está ordenada, ou seja, o algoritmo *bs* é correto.

Repositório: <https://github.com/RafaBonach/bubble-sort-1c1.git>